

Program Description

A program to manage a simple student database that includes operations like adding student details, displaying them, and saving the details to a file.

Algorithm

1. Start.
 2. Define a structure **Student** with fields: **name**, **rollNumber**, **marks**.
 3. Declare an array of **Student** to store student records.
 4. Define functions for:
 - Adding student details.
 - Displaying student details.
 - Saving student details to a file.
 5. Display a menu with options:
 - Add student details.
 - Display all students.
 - Save to file.
 - Exit.
 6. Use a loop to take user input and call the corresponding function based on the choice.
 7. For **Add Student Details**:
 - Accept input for **name**, **rollNumber**, and **marks** from the user.
 - Store the data in the next available slot of the array.
 8. For **Display Students**:
 - Iterate through the array and print each student's details.
 9. For **Save to File**:
 - Open a file in write mode.
 - Write the student records into the file.
 - Close the file.
 10. Repeat the menu until the user chooses "Exit".
 11. End.
-

Pseudocode

```
BEGIN
    DEFINE STRUCTURE Student:
        FIELDS: name[50], rollNumber (int), marks (float)

    DECLARE students[100] AS ARRAY OF Student
    DECLARE count AS INTEGER = 0

    FUNCTION addStudent():
        IF count < 100 THEN
            INPUT name, rollNumber, marks
            STORE details in students[count]
            INCREMENT count
        ELSE
            DISPLAY "Student limit reached."
```

```

        END IF
    END FUNCTION

    FUNCTION displayStudents():
        IF count > 0 THEN
            FOR i = 0 TO count-1 DO
                PRINT students[i].name, students[i].rollNumber,
students[i].marks
            END FOR
        ELSE
            DISPLAY "No students to display."
        END IF
    END FUNCTION

    FUNCTION saveToFile():
        OPEN "students.txt" IN WRITE MODE
        FOR i = 0 TO count-1 DO
            WRITE students[i].name, students[i].rollNumber, students[i].marks TO
FILE
        END FOR
        CLOSE FILE
        DISPLAY "Details saved to file."
    END FUNCTION

    REPEAT
        DISPLAY MENU
        INPUT choice
        SWITCH choice:
            CASE 1: CALL addStudent()
            CASE 2: CALL displayStudents()
            CASE 3: CALL saveToFile()
            CASE 4: EXIT
            DEFAULT: DISPLAY "Invalid option."
        END SWITCH
    UNTIL choice = 4
END

```

C Program

```

#include <stdio.h>
#include <string.h>

// Structure to store student details
struct Student {
    char name[50];
    int rollNumber;
    float marks;
};

// Global variables
struct Student students[100];
int count = 0;

// Function to add a student
void addStudent() {
    if (count < 100) {
        printf("Enter Name: ");
        scanf("%[^\n]", students[count].name);
        printf("Enter Roll Number: ");
        scanf("%d", &students[count].rollNumber);
        printf("Enter Marks: ");
    }
}

```

```

        scanf("%f", &students[count].marks);
        count++;
        printf("Student added successfully.\n");
    } else {
        printf("Student limit reached.\n");
    }
}

// Function to display all students
void displayStudents() {
    if (count > 0) {
        printf("\nStudent Details:\n");
        for (int i = 0; i < count; i++) {
            printf("Name: %s, Roll Number: %d, Marks: %.2f\n",
                students[i].name, students[i].rollNumber, students[i].marks);
        }
    } else {
        printf("No students to display.\n");
    }
}

// Function to save students to a file
void saveToFile() {
    FILE *file = fopen("students.txt", "w");
    if (file == NULL) {
        printf("Error opening file.\n");
        return;
    }
    for (int i = 0; i < count; i++) {
        fprintf(file, "Name: %s, Roll Number: %d, Marks: %.2f\n",
            students[i].name, students[i].rollNumber, students[i].marks);
    }
    fclose(file);
    printf("Details saved to file.\n");
}

// Main function
int main() {
    int choice;

    do {
        printf("\nMenu:\n");
        printf("1. Add Student\n");
        printf("2. Display Students\n");
        printf("3. Save to File\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                addStudent();
                break;
            case 2:
                displayStudents();
                break;
            case 3:
                saveToFile();
                break;
            case 4:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid option. Please try again.\n");
        }
    } while (choice != 4);
}

```

```

    }
} while (choice != 4);

return 0;
}

```

Steps for Algorithm in Lab Exam

1. **Understand the Problem Statement:** Carefully read the given problem and break it into smaller tasks.
2. **Define Inputs and Outputs:** Specify the data your program needs and what it will produce as output.
3. **Write Algorithm:**
 - Use clear and concise language.
 - Number each step logically.
4. **Write Pseudocode:**
 - Translate your algorithm into a structured format closer to code.
 - Use clear indentation and conditional/looping constructs.
5. **Create a Flowchart:**
 - Draw a diagram that visually represents the flow of your algorithm.
 - Use standard symbols for start, process, decision, and end.
6. **Format Properly:** Use consistent indentation, numbering, and spacing for clarity.
7. **Cross-Check:** Validate that your algorithm, pseudocode, and flowchart match the problem statement.

